



TD5-MODELISATION MATHEMATIQUE



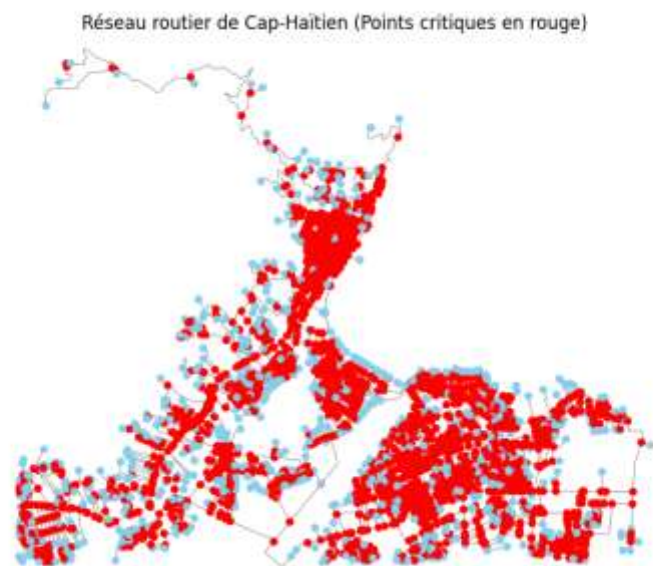
MAY 11, 2026

Etudiante : Christy Gerys LAMBERT

Professeur : Ismaël SAINT ARMOUR

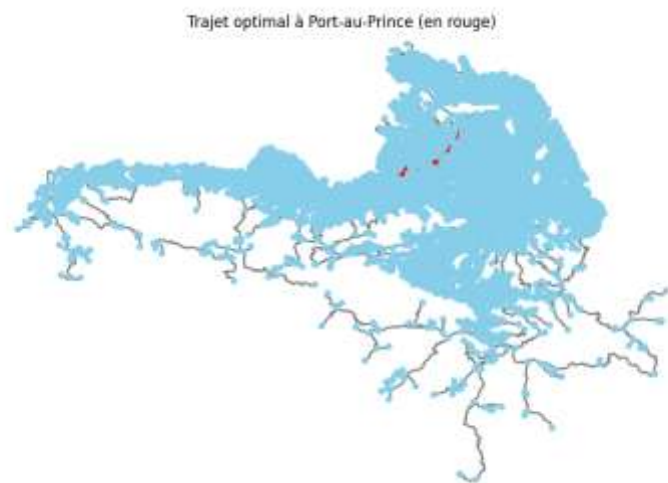
TD1 - Analyse du réseau routier de Cap-Haïtien avec OpenStreetMap

```
C:\Users\Christy> cd tds_1.py > density
1 import osmnx as ox
2 import networkx as nx
3 import matplotlib.pyplot as plt
4 import geopandas as gpd
5
6 place_name = "Cap-Haïtien, Haïti"
7 graph = ox.graph_from_address(place_name, dist=5000, network_type="drive")
8
9 graph = nx.convert_node_labels_to_integers(graph, label_attribute="osmid")
10 graph.remove_nodes_from(list(nx.isolates(graph)))
11
12 density = nx.density(graph)
13 print(f"Densité du réseau : {density:.4f}")
14
15 avg_degree = sum(dict(graph.degree()).values()) / len(graph)
16 print(f"Degré moyen : {avg_degree:.2f}")
17
18 degree_centrality = nx.degree_centrality(graph)
19 top_nodes = sorted(degree_centrality.items(), key=lambda x: x[1], reverse=True)[:5]
20 print("Top 5 nœuds par centralité :", top_nodes)
```



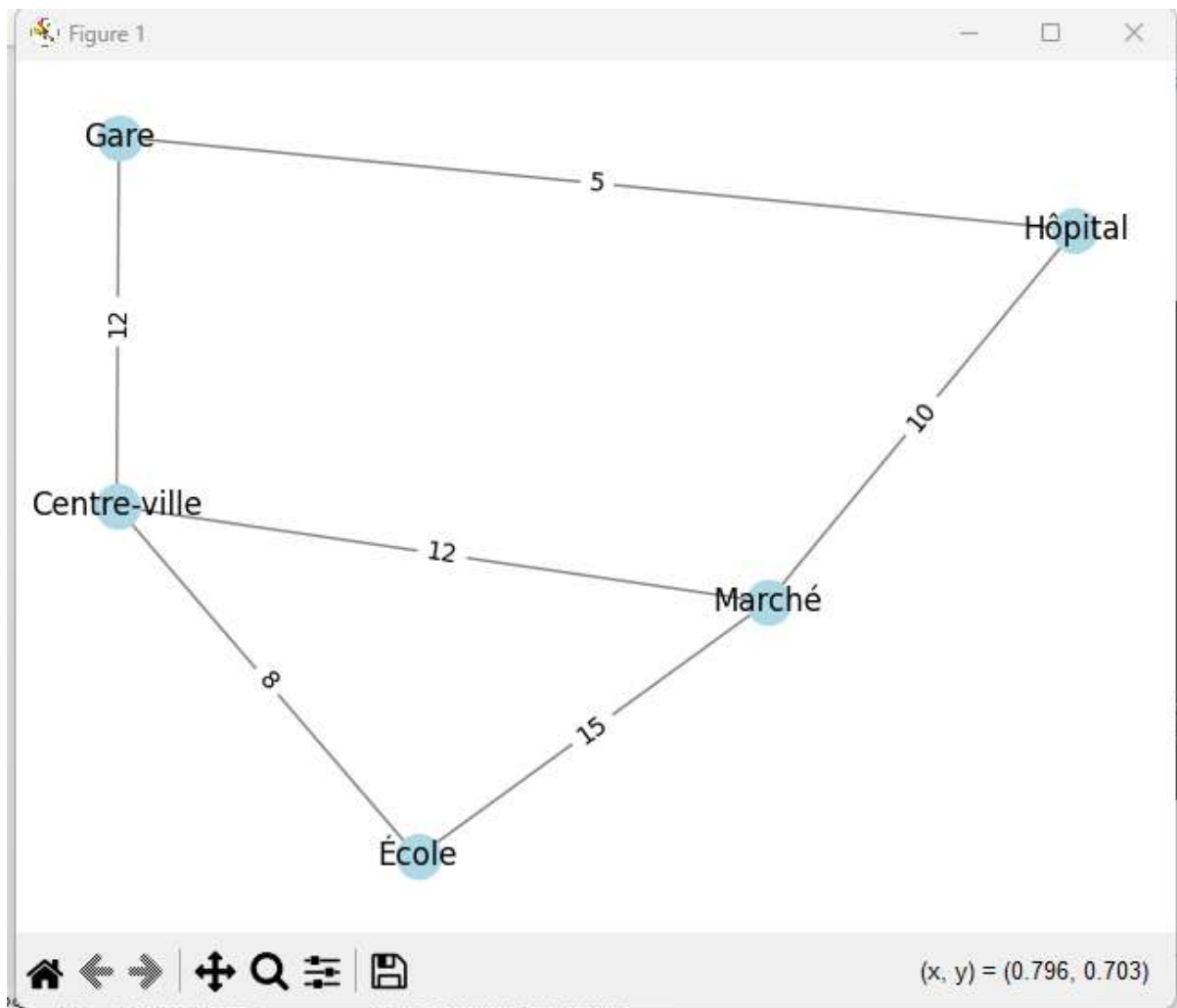
TD2 - Optimisation du transport urbain à Port-au-Prince avec OSMnx

```
1 import osmnx as ox
2 import networkx as nx
3 import matplotlib.pyplot as plt
4
5 place_name = "Port-au-Prince, Haiti"
6 graph = ox.graph_from_place(place_name, network_type="drive")
7 graph.remove_nodes_from(list(nx.isolates(graph)))
8
9 for u, v, data in graph.edges(data=True):
10     data["weight"] = data.get("length", 1)
11
12 orig = (18.5414, -72.3360)
13 dest = (18.5667, -72.3000)
14 orig_node = ox.nearest_nodes(graph, orig[1], orig[0])
15 dest_node = ox.nearest_nodes(graph, dest[1], dest[0])
16
17 shortest_path = nx.shortest_path(graph, orig_node, dest_node, weight="length")
18 shortest_path_length = nx.shortest_path_length(graph, orig_node, dest_node, weight="length")
19
20 speed_kmh = 30
21 time_hours = (shortest_path_length / 1000) / speed_kmh
22 print(f"Distance : {shortest_path_length:.2f} mètres (~{shortest_path_length/1000:.2f} km)")
23 print(f"Temps estimé : {time_hours*60:.2f} minutes")
```



TD3 - Optimisation des transports (tap-tap)

```
1 import networkx as nx
2 import matplotlib.pyplot as plt
3
4 G = nx.Graph()
5
6 stops = ["Marché", "Hôpital", "École", "Gare", "Centre-ville"]
7 G.add_nodes_from(stops)
8
9 edges = [
10     ("Marché", "Hôpital", {"weight": 10}),
11     ("Marché", "École", {"weight": 15}),
12     ("Hôpital", "Gare", {"weight": 5}),
13     ("École", "Centre-ville", {"weight": 8}),
14     ("Gare", "Centre-ville", {"weight": 12}),
15 ]
16 G.add_edges_from(edges)
17
18 shortest_path = nx.shortest_path(G, source="Marché", target="Centre-ville", weight="weight")
19 shortest_time = nx.shortest_path_length(G, source="Marché", target="Centre-ville", weight="weight")
20 print(f"Trajet optimal : {' -> '.join(shortest_path)}")
21 print(f"Temps total : {shortest_time} minutes")
```



```

SyntaxError: invalid syntax
PS C:\Users\Christy> & C:/Users/Christy/AppData/Local/Programs/Python/Python314/python.exe c:/Users/Christy/td5_3.py
Trajet optimal : Marché -> École -> Centre-ville
Temps total : 23 minutes
Nouveau trajet optimal : Marché -> Centre-ville
Nouveau temps : 12 minutes
  
```

TD 4 - Comparaison entre villes

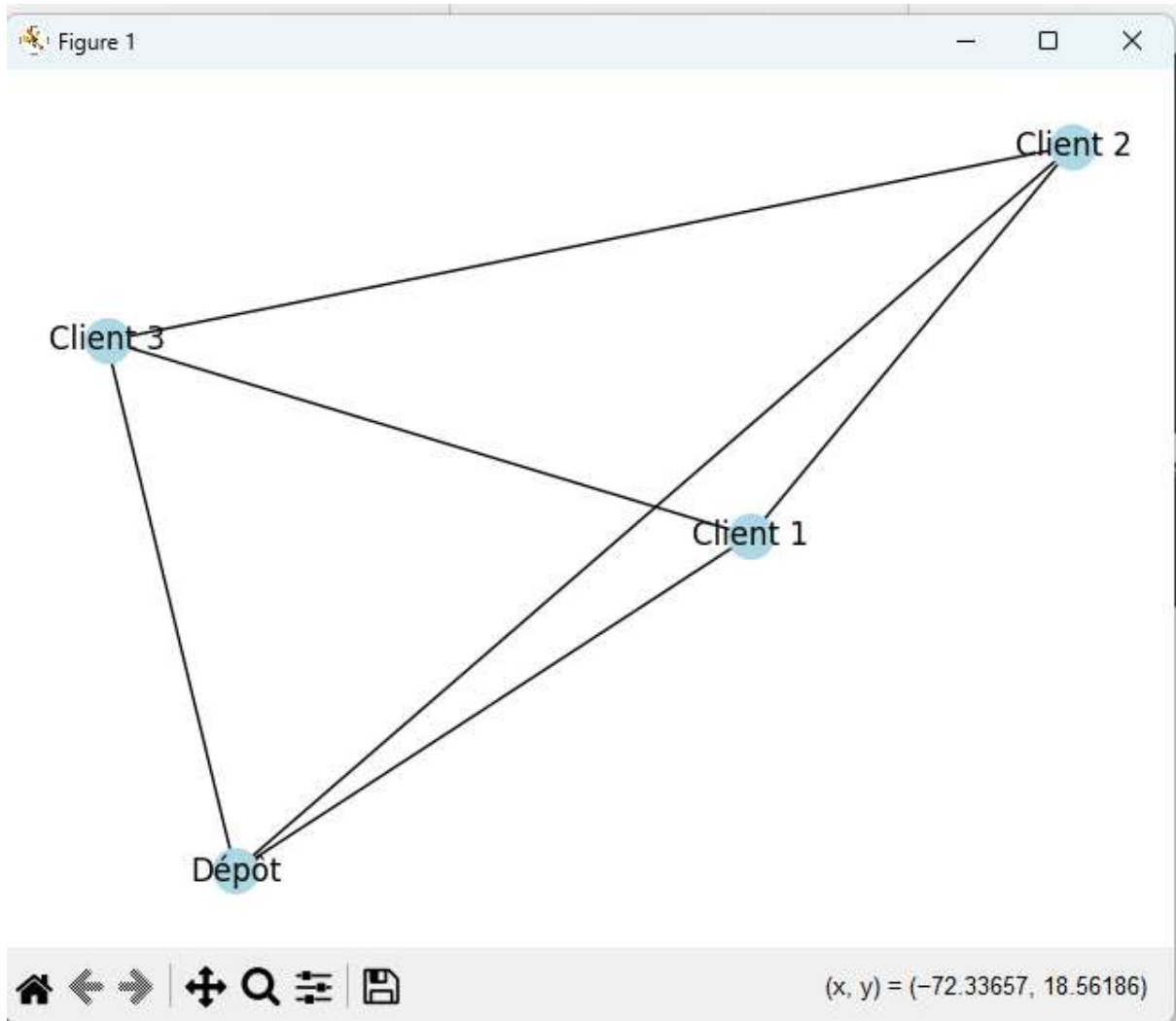
```

C:\Users\Christy> python td5_4.py calculate_metrics
1 import osmnx as ox
2 import networkx as nx
3 import pandas as pd
4 import matplotlib.pyplot as plt
5
6 cap_haitien = ox.graph_from_address("Cap-Haïtien, Haïti", dist=5000, network_type="drive")
7 port-au-prince = ox.graph_from_place("Port-au-Prince, Haïti", network_type="drive")
8
9 cap_haitien.remove_nodes_from(list(nx.isolates(cap_haitien)))
10 port-au-prince.remove_nodes_from(list(nx.isolates(port-au-prince)))
11
12 def calculate_metrics(graph, name):
13     metrics = {
14         "Ville": name,
15         "Nombre de nœuds": len(graph.nodes()),
16         "Nombre de liens": len(graph.edges()),
17         "Densité": nx.density(graph),
18         "Degré moyen": sum(dict(graph.degree()).values()) / len(graph),
19         "Centralité moyenne": sum(nx.degree_centrality(graph).values()) / len(graph),
20     }
21     return metrics
  
```



TD5 -Optimisation de livraison

```
C:\Users\Christy> cd td5_Spy > ...
1 import networkx as nx
2 import matplotlib.pyplot as plt
3 from itertools import permutations
4 delivery_points = {
5     "Dépôt": (18.5414, -72.3360),
6     "Client 1": (18.5500, -72.1200),
7     "Client 2": (18.5600, -72.3100),
8     "Client 3": (18.5550, -72.3400),
9 }
10 G = nx.Graph()
11 for u in delivery_points:
12     for v in delivery_points:
13         if u != v:
14             lat1, lon1 = delivery_points[u]
15             lat2, lon2 = delivery_points[v]
16             distance = ((lat2 - lat1)**2 + (lon2 - lon1)**2)**0.5 * 111
17             G.add_edge(u, v, weight=distance)
18
19
20 nodes = list(delivery_points.keys())
21 min_path = None
22 min_distance = float("inf")
```



```
PS C:\Users\Christy> & C:/Users/Christy/AppData/Local/Programs/Python/Python314/python.exe c:/Users/Christy/td5_5.py
Meilleur parcours : Dépôt -> Client 1 -> Client 2 -> Client 3 -> Dépôt
Distance totale : 8.54 km
Parcours : Dépôt -> Client 3 -> Client 1 -> Client 2 -> Dépôt
Distance : 8.98 km
```

